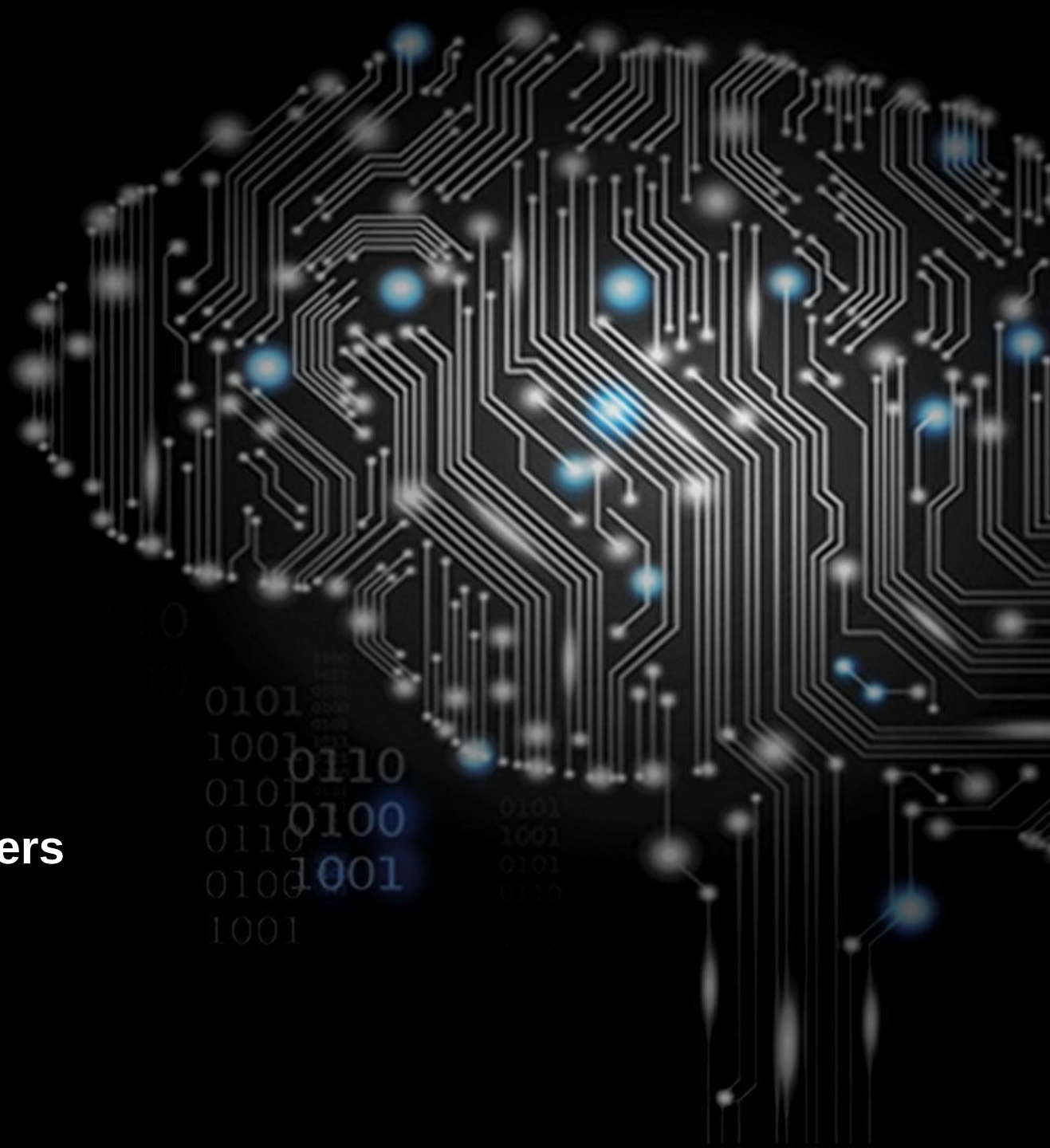


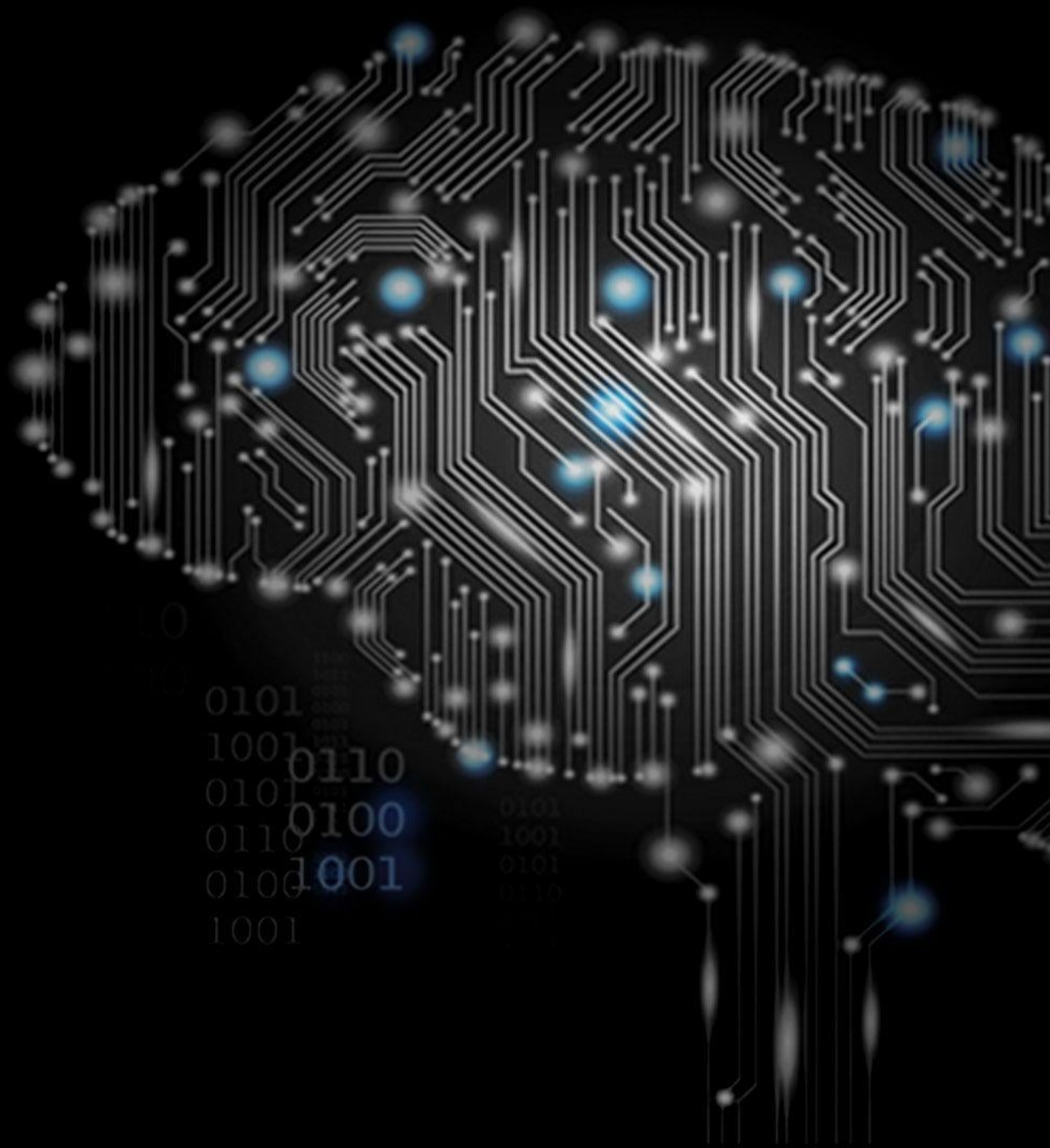
Introduction to Generative AI for Developers

5. Compliance and Guardrails

Sections

- The layered control model
- Grounding on bank guidelines with RAG
- Input guardrails
- Output guardrails
- Implementation options in Java
- Risk framing and audit evidence
- Fine-tuning





Compliance and Guardrails

1. The layered control model

What each tool can and cannot do

- System prompt and brand style guide
  - Sets direction for tone and policy; the model may still deviate
- Grounding in approved content
  - Supplies correct, current facts; does not stop the model from going beyond them
- Guardrails in code or managed services
  - Check and enforce rules on inputs and outputs deterministically or with classifiers; can block, mask, fix or escalate
- Fine-tuning
  - Makes tone, format and narrow task behaviour more consistent; does not teach reliable facts and cannot enforce rules
- Human review
  - Catches what automated layers miss for high-risk outputs; slow and expensive at scale
- Audit log
  - Provides evidence of what happened and why; prevents nothing on its own

The layered control model

- Each request passes through a sequence of controls; each layer catches some failures the others miss

```
Customer message
      |
[1] Input guardrails ......... length limits, PII masking, injection screen, denied topics
      |
[2] Grounding ................ retrieve approved bank content with document IDs
      |
[3] System prompt ............ bank policy, brand voice, task instructions (Unit 3)
      |
[4] Model call ............... structured output, validated schema (Unit 2)
      |
[5] Output guardrails ........ disclaimers, forbidden claims, tone, PII leaks, citations
      |
[6] Human review ............. for high-risk categories or failed checks
      |
[7] Audit log ................ every decision, version and outcome recorded
      |
Reply to customer
```

Mapping risks to layers

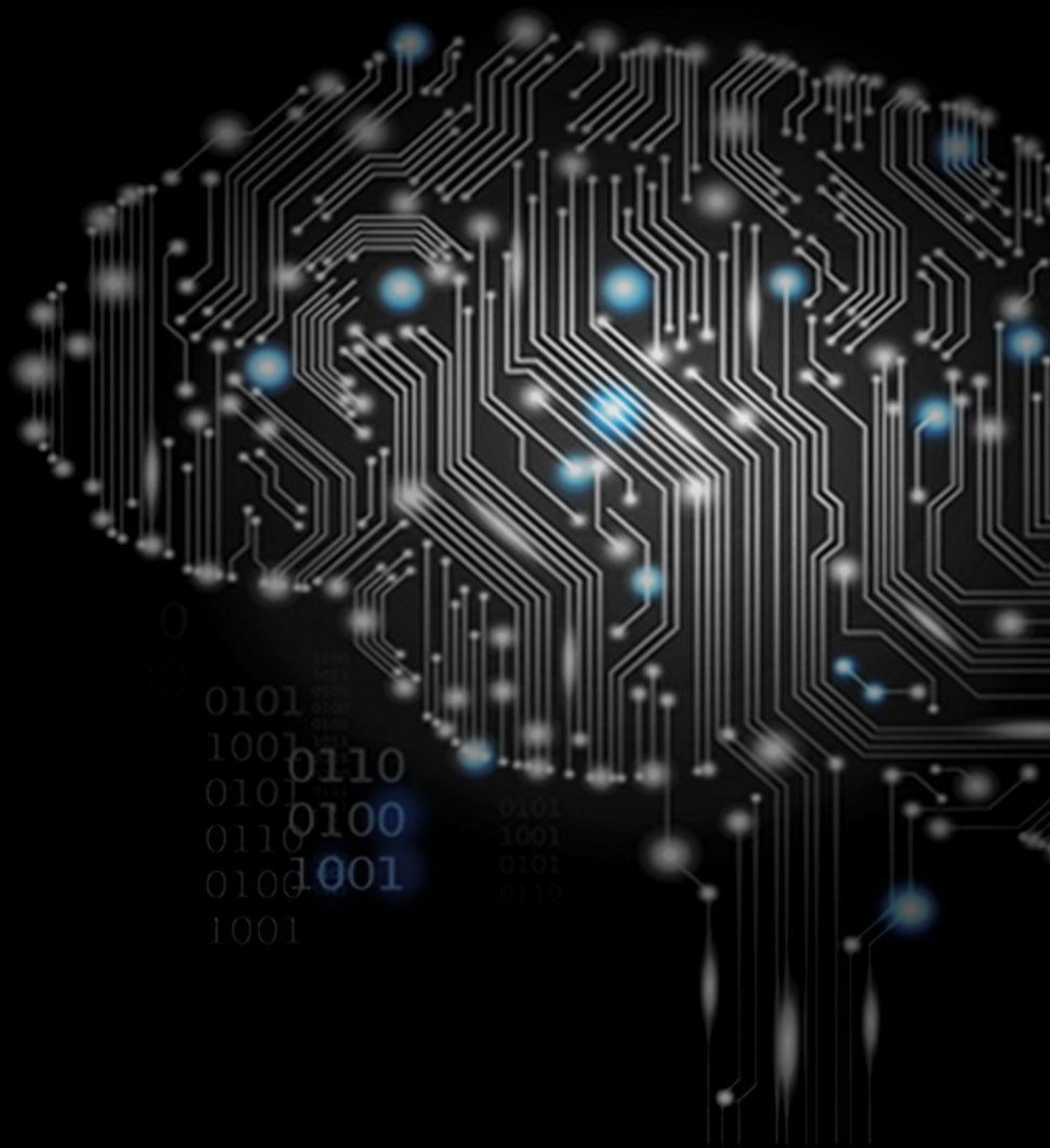
- Customer shares a card number in a message:
  - Input PII masking; output leak check
- Customer tries to make the assistant ignore its rules
  - Ddelimiting (Unit 3), injection screening, output checks
- Customer asks for investment advice: denied-topic input check;
  - Forbidden-claim output check; system prompt rule
- Model states an incorrect fee:
  - Grounding with citations; contextual grounding or fact checks



Mapping risks to layers

- Reply is off-brand or curt
  - System prompt brand voice; tone check; optionally fine-tuning
- Reply omits a required disclaimer
  - Deterministic output rule that adds or requires the text
- Output contains malformed data
  - Schema validation and self-correction (Unit 2)
- Every risk should be covered by at least two independent layers



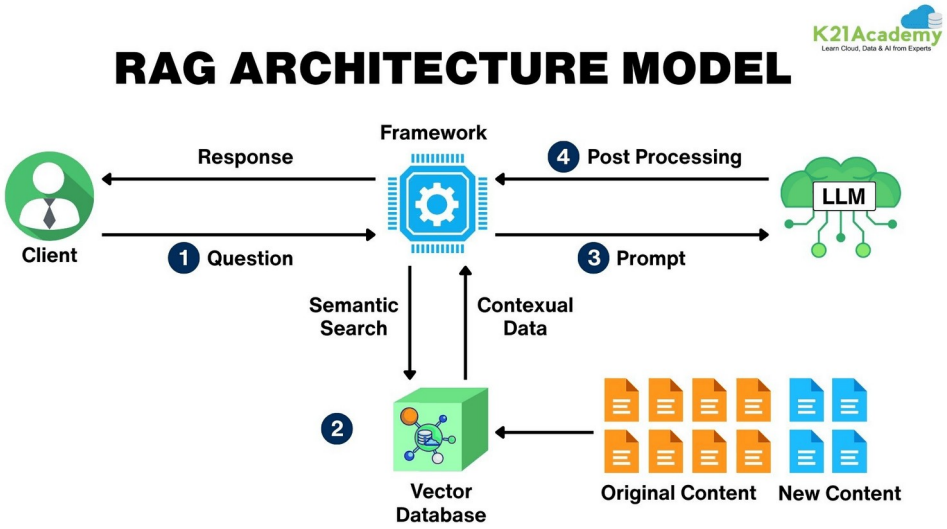


Compliance and Guardrails

3. Grounding on bank guidelines with RAG

Retrieval-augmented generation in brief

- Ingestion (offline)
  - Read approved documents, split them into chunks, embed each chunk, store the vectors with metadata
- Retrieval (per request)
  - Embed the question, find the most similar chunks, optionally filter by metadata
- Augmentation and generation
  - Insert the retrieved chunks into the prompt with their IDs, and instruct the model to answer only from them and cite them (Unit 3)
- Benefits: current, approved facts without retraining; citations for audit
- Costs: an ingestion pipeline to maintain, retrieval quality to measure, more tokens per request



Ingesting bank guidelines

- Spring AI provides an ETL pipeline: document readers, transformers (such as splitters) and vector store writers
- Readers exist for PDF and, through Apache Tika, for Word, PowerPoint, HTML and other formats

```
@Bean
VectorStore vectorStore(EmbeddingModel embeddingModel) {
    return SimpleVectorStore.builder(embeddingModel).build();
}

@Component
class GuidelineIngestion {

    private final VectorStore vectorStore;

    GuidelineIngestion(VectorStore vectorStore) {
        this.vectorStore = vectorStore;
    }

    void ingest(Resource document, String docId, LocalDate effective, String language) {
        List<Document> pages = new TikaDocumentReader(document).read();
        pages.forEach(d -> d.getMetadata().putAll(Map.of(
                "docId", docId, "effective", effective.toString(), "language", language)));
        List<Document> chunks = TokenTextSplitter.builder().build().apply(pages);
        vectorStore.write(chunks);
    }
}
```



Chunking and metadata choices

- Chunk size balances precision against context
  - Small chunks retrieve precisely but can lose surrounding meaning; large chunks carry context but dilute similarity and cost more tokens
  - Spring AI's TokenTextSplitter defaults to chunks of about 800 tokens
- Split along the document's structure where possible: sections, numbered clauses, headings
- Attach metadata to every chunk
  - Document ID and section, effective date, language, product, audience, confidentiality level
- Use metadata to filter at retrieval time
  - Only current versions of a policy, only the customer's language, only content the user is entitled to see
- Re-ingest when documents change, and remove superseded versions



Retrieval at request time

- QuestionAnswerAdvisor retrieves relevant chunks and adds them to the prompt automatically
- Configure how many chunks to retrieve and the minimum similarity
- Apply metadata filters per request, for example by language or product

```
var guidelines = QuestionAnswerAdvisor.builder(vectorStore)
        .searchRequest(SearchRequest.builder()
                .topK(5)
                .similarityThreshold(0.75)
                .build())
        .build();

GroundedAnswer answer = replyClient.prompt()
        .advisors(guidelines)
        .advisors(a -> a.param(QuestionAnswerAdvisor.FILTER_EXPRESSION, "language == 'fr'"))
        .user(u -> u.text(replyTemplate).param("message", customerMessage))
        .call()
        .entity(GroundedAnswer.class);
```



Cited answers

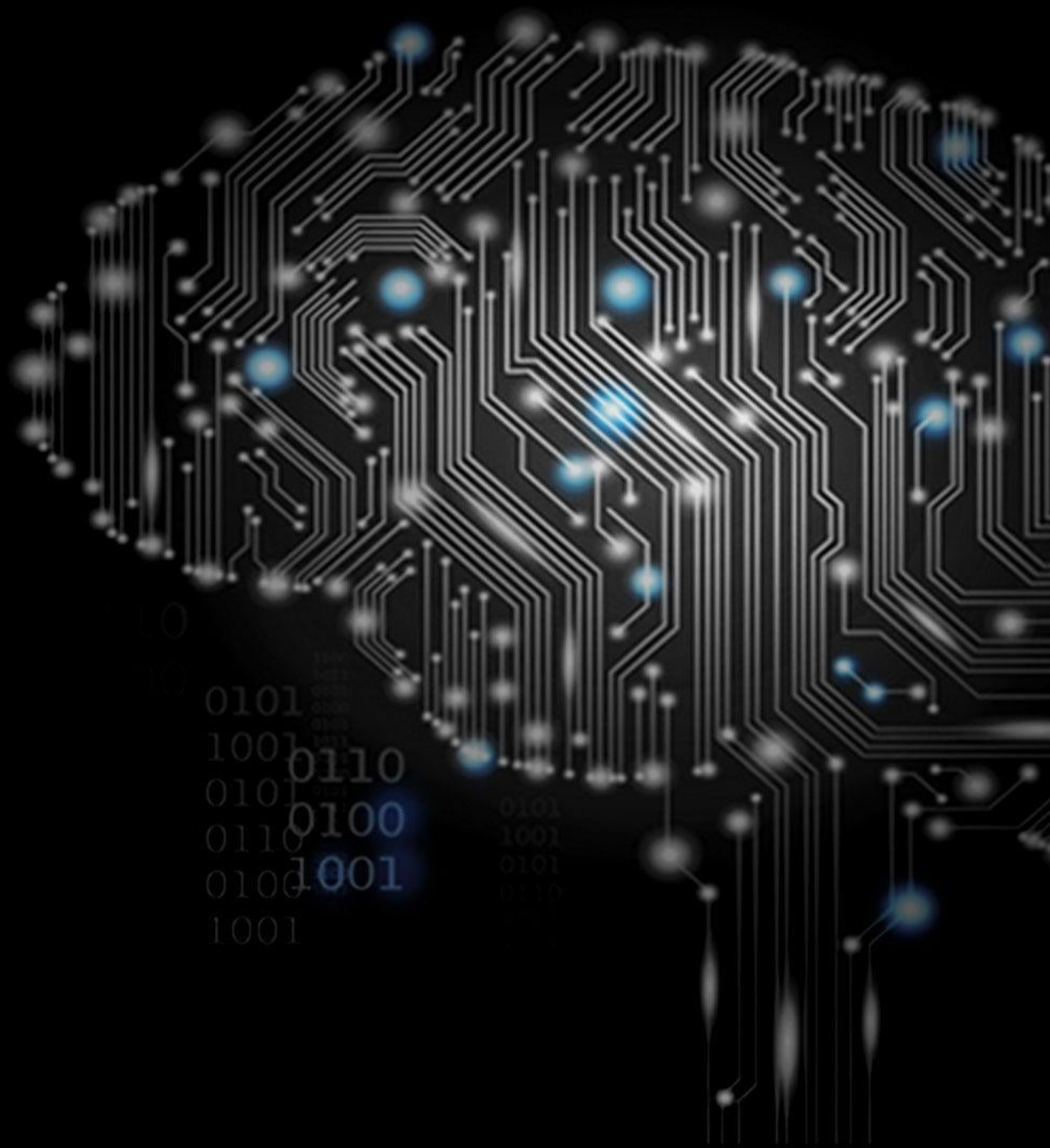
- Include each chunk's document ID and section in the context so the model can cite it
- Use the GroundedAnswer record from Unit 3: answer, cited document IDs, answerable flag
- Check citations in code against the IDs actually retrieved (Unit 3, Slide 3.25)
- Add a grounding check for higher-risk answers
  - Spring AI's FactCheckingEvaluator (Unit 4) or a managed check such as Amazon Bedrock's contextual grounding check
- Log retrieved IDs, cited IDs and the grounding verdict with every response



RAG failure modes

- Stale content: superseded policies remain in the store and are retrieved
- Retrieval misses: the right chunk exists but is not retrieved, so the model answers without it or refuses
- Poor chunking: a rule and its exceptions end up in different chunks
- Poisoned content: a document containing hidden instructions is retrieved and treated as instructions (indirect prompt injection)
- Over-retrieval: too many chunks raise cost and dilute the relevant content
- Mitigations
  - Controlled ingestion of approved documents only, versioning and removal, retrieval quality in the evaluation suite, delimiting retrieved content as data (Unit 3)





Compliance and Guardrails

3. Input guardrails

Input guardrails overview

- Input guardrails run before any model call; they are cheap compared with the model call and reduce risk early
- Typical order
  - Input limits: length, size, rate per user
  - PII detection and masking
  - Prompt-injection screening
  - Off-topic and denied-topic checks
- Each guardrail returns one of these decisions
  - Allow
  - Allow with modified input
  - Block with a reason
- Every decision is logged for audit



PII detection and masking

- Mask personal data before it reaches the model, unless the task genuinely needs it
  - Card numbers, account numbers, Social Insurance Numbers, phone numbers, email and street addresses
- Detection approaches
  - Patterns with validation: regular expressions plus checksums such as the Luhn algorithm for card numbers and SINs
  - Machine-learning detectors for names and addresses, such as the sensitive information filters in Amazon Bedrock Guardrails
- Masking options
  - Replace with a typed placeholder ([CARD_NUMBER]); or tokenize reversibly so the application can restore values after the model call



PII detection and masking

```
class PiiMasker {

    private static final Pattern CARD = Pattern.compile("\\b(?:\\d[ -]?){13,19}\\b");
    private static final Pattern SIN  = Pattern.compile("\\b\\d{3}[ -]?\\d{3}[ -]?\\d{3}\\b");

    String mask(String text) {
        String out = replaceIf(CARD, text, "[CARD_NUMBER]", PiiMasker::luhnValid);
        return replaceIf(SIN, out, "[SIN]", PiiMasker::luhnValid);
    }

    private static String replaceIf(Pattern p, String text, String label, Predicate<String> check) {
        return p.matcher(text).replaceAll(m -> check.test(m.group()) ? label : Matcher.quoteReplacement(m.group()));
    }

    static boolean luhnValid(String candidate) {
        String digits = candidate.replaceAll("\\D", "");
        int sum = 0;
        for (int i = 0; i < digits.length(); i++) {
            int d = digits.charAt(digits.length() - 1 - i) - '0';
            if (i % 2 == 1) { d *= 2; if (d > 9) d -= 9; }
            sum += d;
        }
        return sum % 10 == 0;
    }
}
```



Prompt-injection screening

- Goal: detect inputs that try to override instructions, extract the system prompt or bypass rules
  - Approaches, from cheapest to strongest
    - *Heuristics: known phrases and patterns; easy to evade, useful as a first filter*
    - *Classifier call: a small, fast model asked only whether the input is an injection attempt*
    - *Managed detectors: for example the prompt attack category in Amazon Bedrock Guardrails content filters, covering jailbreaks, injections and prompt leakage*
  - No detector is complete; screening reduces risk and must be combined with delimiting, least-privilege design and output checks

```
public record InjectionVerdict(boolean injectionAttempt, String reason) {}

InjectionVerdict screen(String customerMessage) {
    return screeningClient.prompt()        // small, low-cost model; low reasoning effort
            .user(u -> u.text(screeningTemplate).param("message", customerMessage))
            .call()
            .entity(InjectionVerdict.class);
}
```

Off-topic and denied-topic blocking

- Off-topic: requests outside the assistant's purpose, for example writing a poem or discussing politics
- Denied topics: subjects the bank must not engage with in this channel
  - Personalized investment advice, tax or legal advice, commentary on competitors, other customers' information
- Implementation
  - A topic classifier (rules plus a small model) returning a category from a fixed list
  - Managed denied-topic policies, defined in natural language with examples, in Amazon Bedrock Guardrails
- Respond to blocked topics with an approved, helpful message and a route to the right channel



Off-topic and denied-topic blocking

```
public enum Topic { BANKING_SERVICE, INVESTMENT_ADVICE, TAX_OR_LEGAL, OFF_TOPIC }

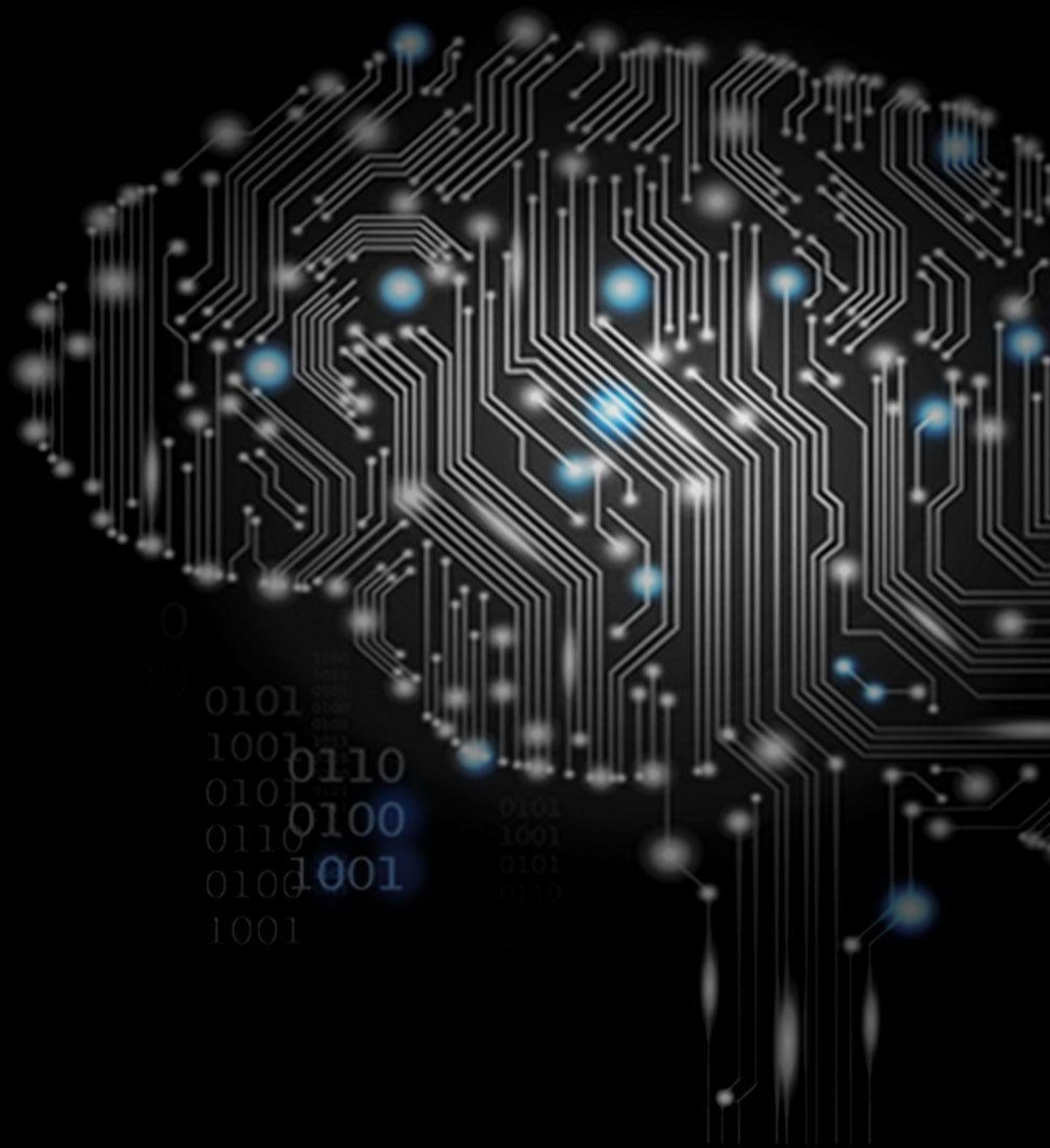
GuardrailDecision checkTopic(String message) {
    Topic topic = topicClient.prompt().user(message).call().entity(Topic.class);
    return switch (topic) {
        case BANKING_SERVICE -> GuardrailDecision.allow();
        case INVESTMENT_ADVICE -> GuardrailDecision.block("denied-topic:investment-advice",
                approvedMessages.get("investment-referral"));
        default -> GuardrailDecision.block("off-topic", approvedMessages.get("off-topic"));
    };
}
```



Input limits

- Maximum message length, enforced before any processing
  - Limits cost, latency and the space available for injection payloads
- Rate limits per customer and per session
  - Protects against abuse and runaway costs (OWASP lists unbounded consumption as a risk)
- Allowed content types and languages for the channel
- Conversation length limits for chat features
- Clear, polite error responses that do not reveal internal rules





Compliance and Guardrails

4. Output guardrails

Output guardrails overview

- Output guardrails run after the model call and before anything reaches the customer or a downstream system
- Typical checks
  - Schema validation and business validation (Unit 2)
  - Required disclaimers and approved wording
  - Forbidden claims and prohibited advice
  - Brand voice and tone
  - Sensitive data leaks and system prompt leakage
  - Citation and grounding checks
- Each check returns pass, fix, retry with feedback, or escalate



Required disclaimers and forbidden claims

- Required disclaimers are deterministic: add them in code, or check that the exact approved wording is present
  - Appending in code is more reliable than asking the model to include them
- Forbidden claims can be partly expressed as rules
  - Phrases such as "guaranteed return", "risk-free", "we promise", specific rate commitments
- Prohibited advice usually needs a classifier, since it can be phrased many ways
- Keep word lists and approved texts in versioned resources, reviewed by compliance

```
class ComplianceRules {

    private static final List<Pattern> FORBIDDEN = List.of(
            Pattern.compile("(?i)\\bguarantee(d)?\\b.*\\breturn"),
            Pattern.compile("(?i)\\brisk[- ]free\\b"),
            Pattern.compile("(?i)\\byou should (buy|sell|invest)\\b"));

    Optional<String> violation(String reply) {
        return FORBIDDEN.stream()
                .filter(p -> p.matcher(reply).find())
                .map(Pattern::pattern)
                .findFirst();
    }

    String withDisclaimer(String reply, Locale locale) {
        return reply + "\n\n" + approvedTexts.disclaimer(locale);
    }
}
```

Required disclaimers and forbidden claims

- Required disclaimers are deterministic: add them in code, or check that the exact approved wording is present
  - Appending in code is more reliable than asking the model to include them
- Forbidden claims can be partly expressed as rules
  - Phrases such as "guaranteed return", "risk-free", "we promise", specific rate commitments
- Prohibited advice usually needs a classifier, since it can be phrased many ways
- Keep word lists and approved texts in versioned resources, reviewed by compliance

```
class ComplianceRules {

    private static final List<Pattern> FORBIDDEN = List.of(
            Pattern.compile("(?i)\\bguarantee(d)?\\b.*\\breturn"),
            Pattern.compile("(?i)\\brisk[- ]free\\b"),
            Pattern.compile("(?i)\\byou should (buy|sell|invest)\\b"));

    Optional<String> violation(String reply) {
        return FORBIDDEN.stream()
                .filter(p -> p.matcher(reply).find())
                .map(Pattern::pattern)
                .findFirst();
    }

    String withDisclaimer(String reply, Locale locale) {
        return reply + "\n\n" + approvedTexts.disclaimer(locale);
    }
}
```

Brand voice and tone checks

- Tone is hard to check with rules; use an LLM judge with a rubric (the BrandToneEvaluator from Unit 4)
- Decide where to apply it
  - On every reply: highest assurance, highest cost and latency
  - On a sample, or only on replies flagged by cheaper checks: lower cost, with monitoring of the sampled rate
- Include specific, binary criteria: polite, plain language, same language as the customer, no blame, no jargon
- Calibrate the judge against compliance reviewers' judgements before relying on it



Sensitive-data leak checks

- Scan outputs for the same PII patterns masked on input
  - Catches values that came from retrieved documents, tool results or the model's own invention
- Check for system prompt leakage: distinctive phrases from the system prompt appearing in replies
- Check that any account or customer identifiers in the output belong to the current customer
- Block and escalate on detection; never send the unredacted text to logs that lack the right access controls

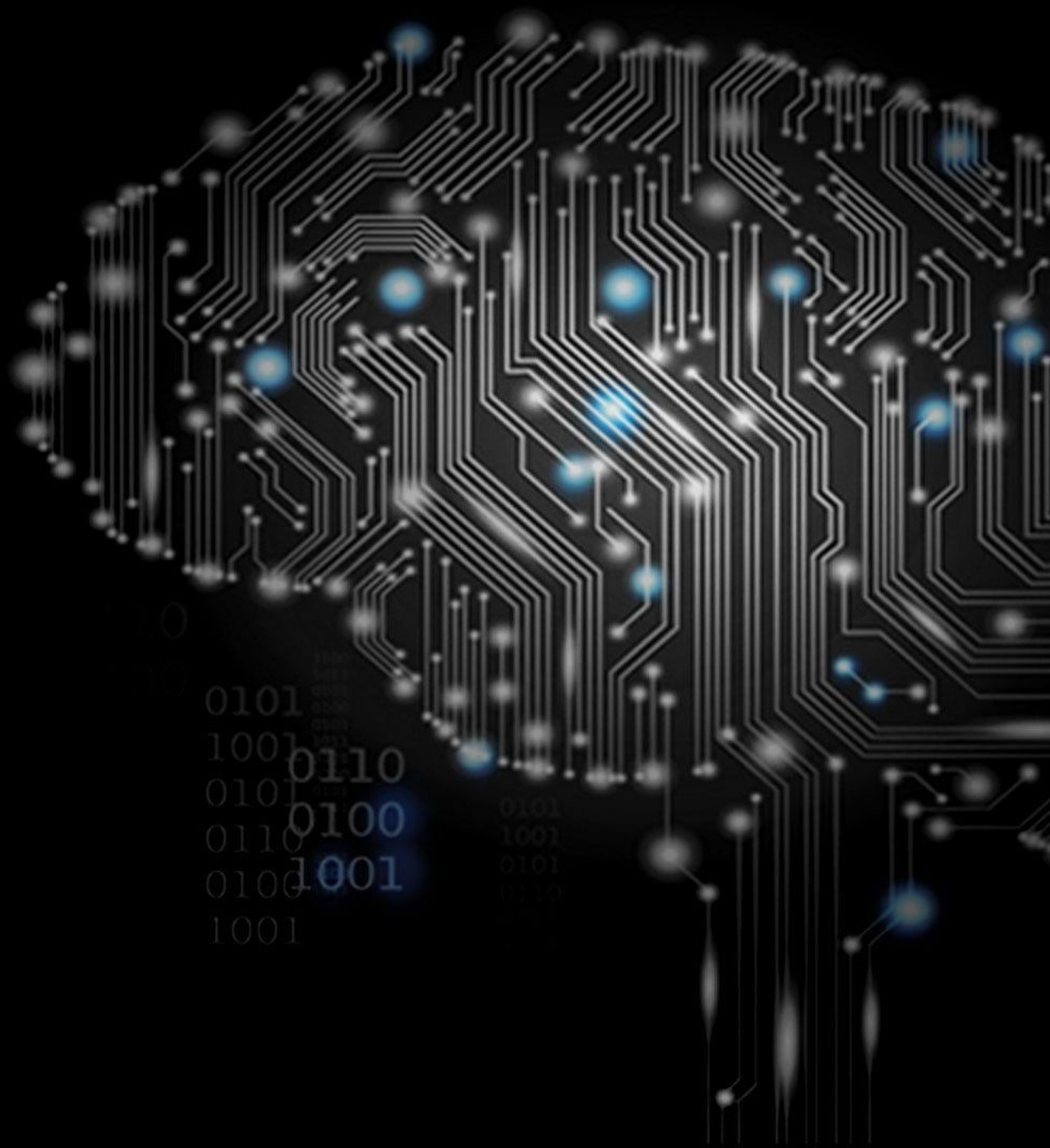
```
Optional<String> leak(String reply, String systemPrompt) {
    if (!piiMasker.mask(reply).equals(reply)) return Optional.of("pii-in-output");
    if (canaryPhrases(systemPrompt).stream().anyMatch(reply::contains)) return Optional.of("system-prompt-leak");
    return Optional.empty();
}
```



When a guardrail fires

- Choose the response per rule, in advance
  - Fix: deterministic corrections such as appending a disclaimer
  - Retry with feedback: re-prompt once, stating the violation
  - Block: return an approved fallback message
  - Escalate: route to a human agent with the context
- Limit automatic retries, usually to one; repeated failures indicate a prompt or data problem
- Streaming complicates output guardrails, since text may already be on the customer's screen
  - For customer-facing replies that need checks, generate fully, check, then send; or stream only to internal staff
- Record which guardrail fired, why, and what action was taken





Compliance and Guardrails

5. Implementation options in Java

Guardrails as Spring AI advisors

- Advisors intercept every request and response passing through a ChatClient
- A CallAdvisor can modify the request, inspect the response, or stop the chain and return its own response
- Lower order values run first on the request and last on the response

```
public class PiiMaskingAdvisor implements CallAdvisor {

    private final PiiMasker masker;

    public PiiMaskingAdvisor(PiiMasker masker) {
        this.masker = masker;
    }

    @Override
    public String getName() { return "pii-masking"; }

    @Override
    public int getOrder() { return Ordered.HIGHEST_PRECEDENCE + 100; }

    @Override
    public ChatClientResponse adviseCall(ChatClientRequest request, CallAdvisorChain chain) {
        String original = request.prompt().getUserMessage().getText();
        String masked = masker.mask(original);
        ChatClientRequest safe = request.mutate()
                .prompt(request.prompt().augmentUserMessage(masked))
                .build();
        return chain.nextCall(safe);
    }
}
```



An output guardrail advisor

- Call the rest of the chain, inspect the result, then pass, fix, retry once or block
- Blocking means returning a response without calling the model again, with an approved fallback message

```
public class ComplianceOutputAdvisor implements CallAdvisor {

    @Override public String getName() { return "compliance-output"; }
    @Override public int getOrder() { return Ordered.HIGHEST_PRECEDENCE + 200; }

    @Override
    public ChatClientResponse adviseCall(ChatClientRequest request, CallAdvisorChain chain) {
        ChatClientResponse response = chain.nextCall(request);
        String text = response.chatResponse().getResult().getOutput().getText();

        Optional<String> violation = rules.violation(text).or(() -> leaks.leak(text, systemPrompt))
        if (violation.isEmpty()) {
            audit.record("compliance-output", "pass", request);
            return response;
        }
        audit.record("compliance-output", "block:" + violation.get(), request);
        return fallbackResponse(approvedMessages.get("specialist-follow-up"), request);
    }
}
```



Ordering and built-in advisors

- Order guardrail advisors deliberately
  - Input masking and screening first; retrieval next; output checks wrapping the model call; logging outermost
- Spring AI includes advisors relevant to this unit
  - QuestionAnswerAdvisor and RetrievalAugmentationAdvisor for grounding
  - SafeGuardAdvisor, a simple advisor that blocks requests containing configured sensitive words
  - StructuredOutputValidationAdvisor for schema validation and self-correction (Unit 2)
  - SimpleLoggerAdvisor for request and response logging
- Built-in advisors cover common cases; bank-specific rules need custom advisors

```
@Bean
ChatClient replyClient(ChatClient.Builder builder, VectorStore store, GuardrailBeans g) {
    return builder
            .defaultAdvisors(
                    g.piiMasking(),                                  // first on the way in
                    g.injectionScreen(),
                    g.topicCheck(),
                    QuestionAnswerAdvisor.builder(store).build(),
                    g.complianceOutput())                            // inspects the response
            .build();
}
```



LangChain4j guardrails

- LangChain4j provides first-class InputGuardrail and OutputGuardrail interfaces for its AI Services
- Output guardrail results express the actions directly
  - success(), successWith(rewritten), failure(...), fatal(...), retry(...), reprompt(message, newPrompt)
  - The maximum number of retries is configurable
- Guardrails are attached declaratively to AI Service methods

```
public class DisclaimerGuardrail implements OutputGuardrail {

    @Override
    public OutputGuardrailResult validate(AiMessage response) {
        String text = response.text();
        if (rules.violation(text).isPresent()) {
            return reprompt("Forbidden claim detected",
                    "Rewrite the reply without promises about returns or outcomes.");
        }
        return successWith(text + "\n\n" + approvedTexts.disclaimer(Locale.CANADA));
    }
}

interface ReplyAssistant {
    @InputGuardrails(PiiMaskingInputGuardrail.class)
    @OutputGuardrails(DisclaimerGuardrail.class)
    String draftReply(@UserMessage String customerMessage);
}
```



Managed guardrails: Amazon Bedrock Guardrails

- A configurable guardrail service with several policy types
  - Content filters, including a prompt attack category for jailbreaks, injections and prompt leakage
  - Denied topics defined in natural language
  - Word filters for exact words and phrases
  - Sensitive information filters: ML-based PII detection and custom regex, with block or mask actions
  - Contextual grounding checks to detect answers not supported by source passages
  - Automated reasoning checks that validate responses against logical policies
- Can be attached to Bedrock model calls, or called independently through the ApplyGuardrail API, including for models hosted elsewhere
- Considerations
  - Configured in AWS, so changes follow the cloud change-management process and bypass code review unless governed separately
  - Blocked content appears in plain text in Bedrock model invocation logs if those logs are enabled



Calling ApplyGuardrail from the AWS SDK for Java

- The Bedrock Runtime client exposes applyGuardrail for checking any text against a configured guardrail
- Specify whether the text is an input or an output, so the right policies apply
- The response says whether the guardrail intervened and provides masked text where applicable

```
ApplyGuardrailResponse result = bedrockRuntime.applyGuardrail(r -> r
        .guardrailIdentifier(guardrailId)
        .guardrailVersion("1")
        .source(GuardrailContentSource.OUTPUT)
        .content(GuardrailContentBlock.fromText(
                GuardrailTextBlock.builder().text(draftReply).build())));

if (result.action() == GuardrailAction.GUARDRAIL_INTERVENED) {
    String safeText = result.outputs().getFirst().text();   // masked or replacement text
    audit.record("bedrock-guardrail", "intervened", result.assessments());
    return safeText;
}
return draftReply;
```



Calling ApplyGuardrail from the AWS SDK for Java

- The Bedrock Runtime client exposes applyGuardrail for checking any text against a configured guardrail
- Specify whether the text is an input or an output, so the right policies apply
- The response says whether the guardrail intervened and provides masked text where applicable

```
ApplyGuardrailResponse result = bedrockRuntime.applyGuardrail(r -> r
        .guardrailIdentifier(guardrailId)
        .guardrailVersion("1")
        .source(GuardrailContentSource.OUTPUT)
        .content(GuardrailContentBlock.fromText(
                GuardrailTextBlock.builder().text(draftReply).build())));

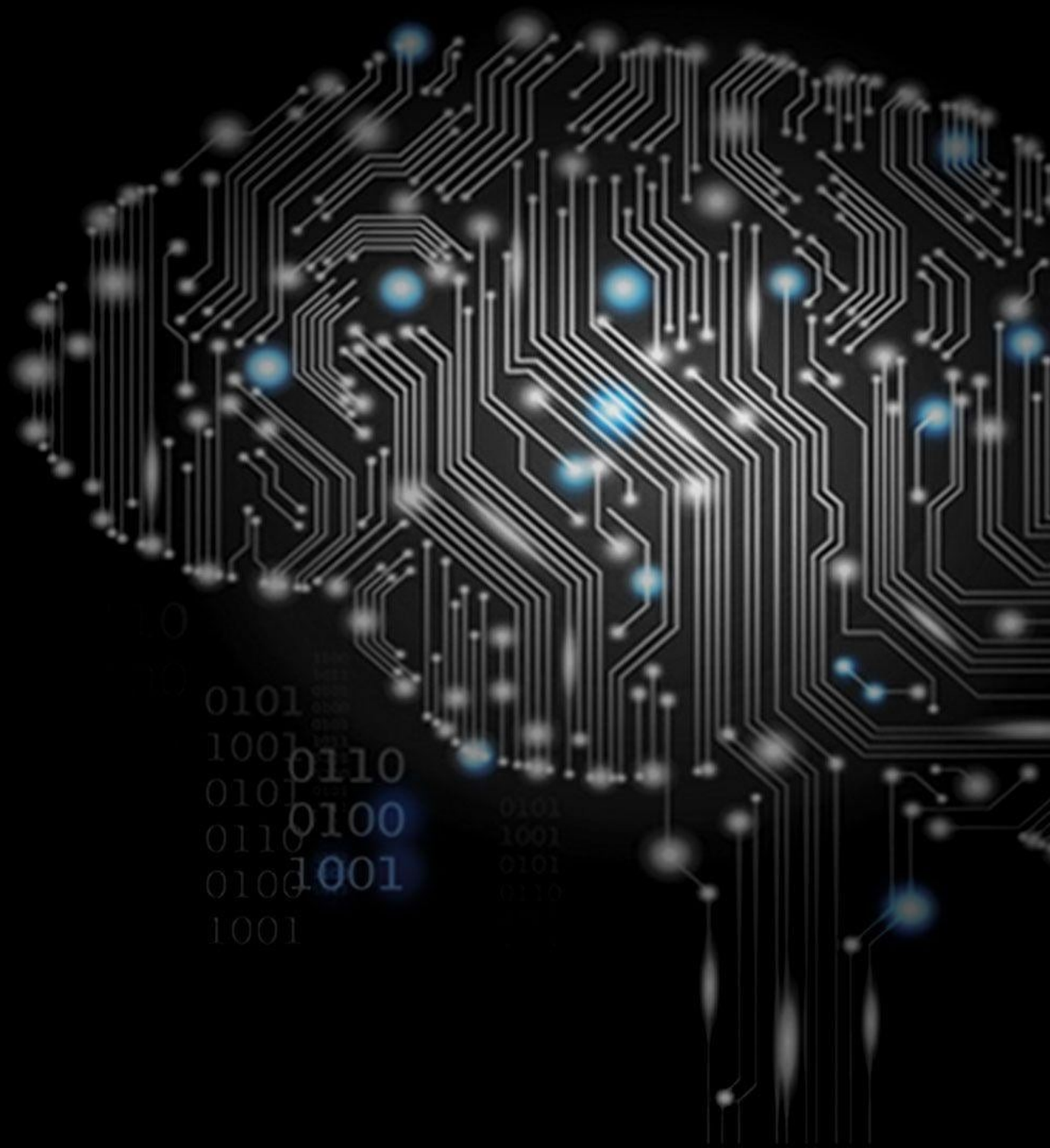
if (result.action() == GuardrailAction.GUARDRAIL_INTERVENED) {
    String safeText = result.outputs().getFirst().text();   // masked or replacement text
    audit.record("bedrock-guardrail", "intervened", result.assessments());
    return safeText;
}
return draftReply;
```



Choosing an implementation

- Custom code in Spring AI advisors
  - Full control, testable with the Day 1 suite, reviewed like code; the team builds and maintains detectors
- LangChain4j guardrails
  - Clear guardrail semantics with retry and reprompt built in; best where LangChain4j is already the stack
- Managed services such as Amazon Bedrock Guardrails
  - Ready-made detectors for PII, prompt attacks and grounding; additional latency, per-use cost, cloud-side configuration and data residency questions
- Common combination
  - Deterministic rules in code (disclaimers, patterns, schemas) plus a managed or model-based detector for fuzzy categories (PII names, injection, topics)
- Whatever the choice, test guardrails with the evaluation suite, including adversarial cases





Compliance and Guardrails

6. Risk framing and audit evidence

Testing guardrails

- Every guardrail needs positive and negative tests
  - Inputs that must be blocked or masked, and similar inputs that must pass
- Extend the Unit 4 golden dataset with guardrail and red-team cases
  - Injection attempts, PII in many formats, denied-topic phrasings in English and French, forbidden claims
- Measure false positives as carefully as false negatives
  - Blocking legitimate customers has real cost and reputational impact
- Run guardrail tests in the CI gate; a guardrail change is a behaviour change



OWASP Top 10 for LLM Applications (2025)

- LLM01 Prompt Injection: delimiting, injection screening, least privilege, output checks
- LLM02 Sensitive Information Disclosure: PII masking, output leak checks, retrieval access filters
- LLM05 Improper Output Handling: schema validation, treating output as untrusted in downstream code
- LLM06 Excessive Agency: least-privilege tools, human approval
- LLM07 System Prompt Leakage: no secrets in prompts, canary checks
- LLM08 Vector and Embedding Weaknesses: controlled ingestion, metadata access control
- LLM09 Misinformation: grounding, citations, grounding checks
- LLM10 Unbounded Consumption: input limits, rate limits, cost alerts
- The remaining items (LLM03 Supply Chain, LLM04 Data and Model Poisoning) concern model and data sourcing, relevant to fine-tuning and open-weight model



OWASP Top 10 for Agentic Applications (2026)

- ASI01 Agent Goal Hijack
- ASI02 Tool Misuse and Exploitation
- ASI03 Identity and Privilege Abuse
- ASI04 Agentic Supply Chain Vulnerabilities
- ASI05 Unexpected Code Execution
- ASI06 Memory and Context Poisoning
- ASI07 Insecure Inter-Agent Communication
- ASI08 Cascading Failures
- ASI09 Human-Agent Trust Exploitation
- ASI10 Rogue Agents
- The LLM list still applies to agents; the agentic list adds risks from autonomy, tools and memory



Model risk management and audit evidence

- OSFI Guideline E-23 (in force from May 2027) applies model risk management to AI models (Unit 1)
- Evidence the development team can provide
  - Design: the layered control model, with the risks each layer addresses
  - Validation: golden-set results, guardrail test results, red-team results, before and after each change
  - Monitoring: guardrail trigger rates, refusal rates, grounding failures, cost (Unit 6)
  - Change records: prompt, model, guardrail and dataset versions with reviews and approvals
- Write an audit record for every guardrail decision

```
public record GuardrailAuditRecord(
        Instant timestamp,
        String correlationId,
        String guardrail,          // e.g. "pii-masking", "denied-topic"
        String decision,           // allow, mask, block, retry, escalate
        String reason,
        String promptVersion,
        String model,
        String guardrailVersion,
        List<String> retrievedDocIds) {}
```

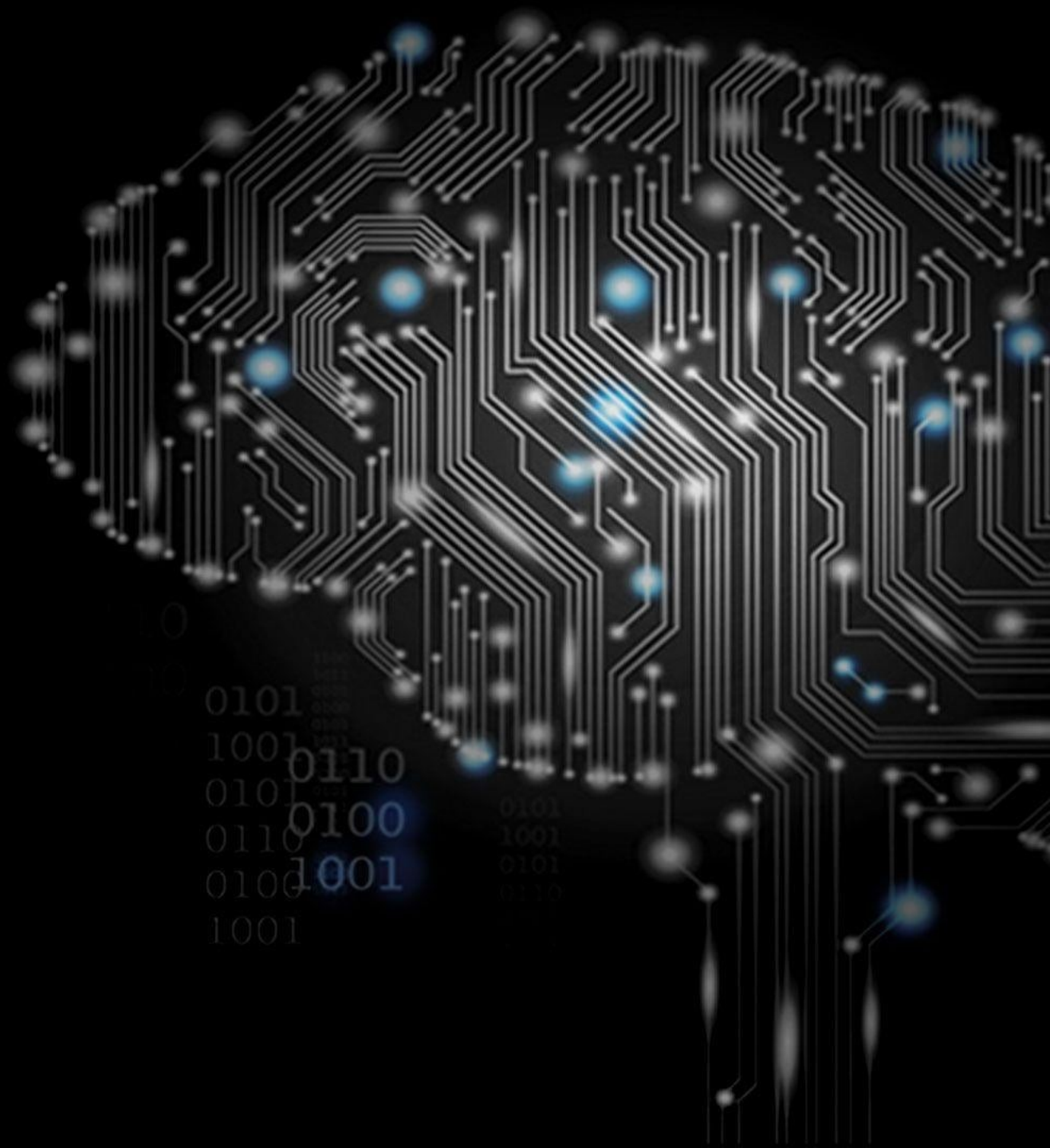


Model risk management and audit evidence

- OSFI Guideline E-23 (in force from May 2027) applies model risk management to AI models (Unit 1)
- Evidence the development team can provide
  - Design: the layered control model, with the risks each layer addresses
  - Validation: golden-set results, guardrail test results, red-team results, before and after each change
  - Monitoring: guardrail trigger rates, refusal rates, grounding failures, cost (Unit 6)
  - Change records: prompt, model, guardrail and dataset versions with reviews and approvals
- Write an audit record for every guardrail decision

```
public record GuardrailAuditRecord(
        Instant timestamp,
        String correlationId,
        String guardrail,          // e.g. "pii-masking", "denied-topic"
        String decision,           // allow, mask, block, retry, escalate
        String reason,
        String promptVersion,
        String model,
        String guardrailVersion,
        List<String> retrievedDocIds) {}
```





Compliance and Guardrails

7. Fine-tuning

What fine-tuning is

- Fine-tuning continues training a pre-trained model on the bank's own examples, producing a customized model
- Supervised fine-tuning (SFT)
  - Train on example inputs with the desired outputs; teaches format, tone and task behaviour
- Preference fine-tuning, such as Direct Preference Optimization (DPO)
  - Train on pairs of a preferred and a less preferred response to the same input; teaches the model which style to favour
  - OpenAI recommends SFT first, then DPO starting from the SFT model
- Reinforcement fine-tuning (RFT)
  - The model generates answers and a grader scores them; training increases the likelihood of high-scoring answers
  - Suited to tasks with clearly gradable outcomes



When fine-tuning pays off, and when it does not

- Good fits
  - Consistent brand voice and reply structure across very high volumes
  - Strict output formats that prompting alone does not hold reliably
  - Narrow, stable tasks such as classification, where a tuned small model can match a large one at lower cost and latency
  - Shorter prompts, since behaviour no longer needs long instructions and examples
- Poor fits
  - Facts: product details, fees and policies change and must be cited; use grounding
  - Changing policy: every change would require retraining and revalidation
  - Enforcement: a tuned model can still produce forbidden content; guardrails remain necessary
- Try prompting, examples and grounding first; fine-tune only when evaluation shows a gap they cannot close



Which models can be tuned (as of October 2026)

- OpenAI API
  - OpenAI is winding down its self-serve fine-tuning platform: closed to new users; existing eligible customers can create training jobs until January 6, 2027
  - Supported base models are the GPT-4.1 family (gpt-4.1, gpt-4.1-mini, gpt-4.1-nano); the nano snapshot shut down on October 23, 2026, and the others shut down on January 14, 2027
- Microsoft Foundry
  - Lists SFT and DPO for GPT-4o and GPT-4.1 models, RFT for o4-mini and GPT-5 (GPT-5 by invitation), and SFT for some open-weight models such as Llama 3.3 70B, Qwen 32B and gpt-oss-20b
  - Standard training regions listed are in the United States and Sweden, which raises data residency questions for Canadian customer data



Which models can be tuned (as of October 2026)

- Amazon Bedrock
  - Supports customization for selected models, including reinforcement fine-tuning for Amazon Nova 2 Lite and fine-tuning of open-weight models through OpenAI-compatible APIs
- On-premises
  - Open-weight models can be tuned in-house with parameter-efficient methods such as LoRA; requires GPU capacity and ML engineering skills
- The current frontier models used elsewhere in this course cannot be fine-tuned through their providers' APIs



Preparing training data

- Training data is JSON Lines: one example per line, in chat-message format
- Supervised examples contain the system message, a user message and the ideal assistant reply
  - At least 10 examples are required; providers suggest starting with about 50 high-quality examples and evaluating before adding more
  - The system message used in production should match the one used in training
- Preference examples contain an input plus a preferred and a non-preferred reply
- Data quality matters more than volume
  - Replies written or approved by brand and compliance teams; no real customer PII; balanced across categories and languages



Preparing training data

```
{"messages":[{"role":"system","content":"You draft replies for Northwind Bank."},
  {"role":"user","content":"My card was declined at the grocery store."},
  {"role":"assistant","content":"I'm sorry for the trouble at the checkout. I can see this can be frustrating..."}]}
```

```
{"input":{"messages":[{"role":"user","content":"Why was I charged a fee?"}]},
 "preferred_output":[{"role":"assistant","content":"Thanks for asking. The $2.50 fee applies to..."}],
 "non_preferred_output":[{"role":"assistant","content":"Fees are in the agreement you signed."}]}
```

```
// Writing JSONL from approved examples
try (var out = Files.newBufferedWriter(Path.of("train.jsonl"))) {
    for (ApprovedExample ex : approvedExamples) {
        out.write(jsonMapper.writeValueAsString(Map.of("messages", List.of(
                Map.of("role", "system", "content", SYSTEM_PROMPT),
                Map.of("role", "user", "content", ex.customerMessage()),
                Map.of("role", "assistant", "content", ex.approvedReply())))));
        out.newLine();
    }
}
```



The fine-tuning workflow and its evaluation

- Baseline: run the Day 1 evaluation suite on the current prompt and base model, and record the scores
- Prepare and split data: training set, validation set, and the golden set kept separate from training
- Train, starting small, and inspect training and validation metrics
- Evaluate the tuned model with the same suite, the same thresholds and the same guardrail tests
  - Compare quality, tone scores, refusal rates, latency and cost per request
- Deploy behind the same guardrails, switchable by configuration so rollback is immediate
- Monitor in production and retrain only through the same governed process



Cost considerations for fine-tuning

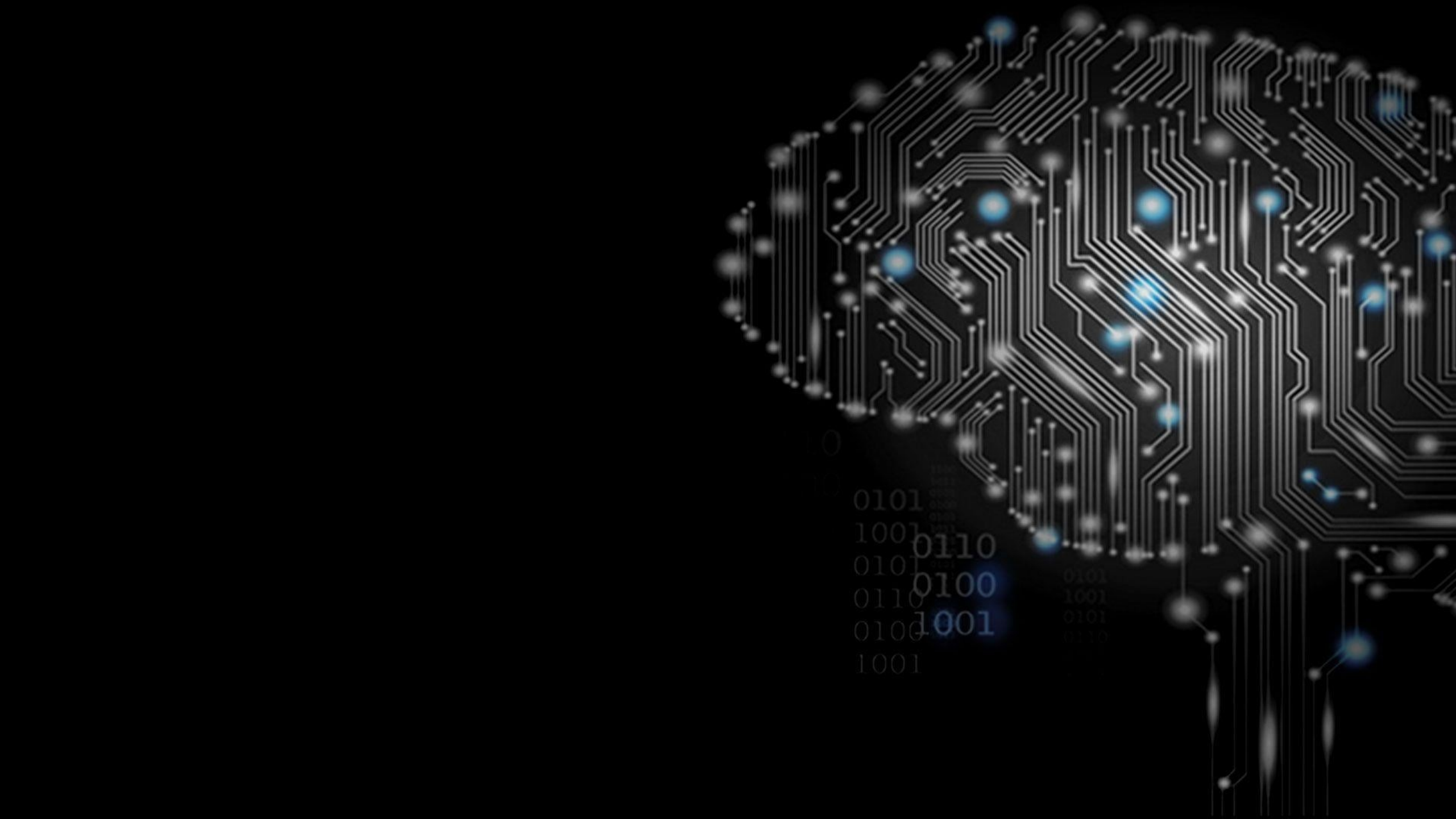
- One-time costs
  - Preparing and approving training data, often the largest cost because it needs expert time
  - Training compute, billed per training token or per hour depending on the platform
- Ongoing costs
  - Hosting or inference charges for the custom model, which may differ from base-model pricing
  - Re-validation and retraining when policies, products or base models change
  - Migration when the base model is retired, as OpenAI's shutdown dates illustrate
- Possible savings
  - Shorter prompts and smaller models for high-volume tasks
- Decide with a cost comparison over the expected lifetime, using measured token counts from Unit 2



Cost considerations for fine-tuning

- One-time costs
  - Preparing and approving training data, often the largest cost because it needs expert time
  - Training compute, billed per training token or per hour depending on the platform
- Ongoing costs
  - Hosting or inference charges for the custom model, which may differ from base-model pricing
  - Re-validation and retraining when policies, products or base models change
  - Migration when the base model is retired, as OpenAI's shutdown dates illustrate
- Possible savings
  - Shorter prompts and smaller models for high-volume tasks
- Decide with a cost comparison over the expected lifetime, using measured token counts from Unit 2




0101
1001
0110
0100
1001